

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ІІІ

Звіт

з лабораторної роботи №4 з дисципліни
«Системне програмне забезпечення»

„Масиви”

Виконав ІІ-45 Янов Б. Є.

Перевірив Бабич Б. Б.

Київ 2026

Лабораторна робота №4

Масиви

Завдання:

1. Написати програму, яка повинна мати наступний функціонал:
 - Можливість введення користувачем розміру одномірного масиву.
 - Можливість введення користувачем значень елементів одномірного масиву.
 - Можливість знаходження суми елементів одномірного масиву.
 - Можливість пошуку максимального (або мінімального) елемента одномірного масиву.
 - Можливість сортування одномірного масиву цілих чисел загального вигляду.
2. Написати програму, яка буде мати наступний функціонал:
 - Можливість введення користувачем розміру двомірного масиву.
 - Можливість введення користувачем значень елементів двомірного масиву.
 - Можливість пошуку координат всіх входжень заданого елемента в двомірному масиві, елементи масиву та пошуковий елемент вводить користувач.
3. Програма повинна мати захист від некоректного введення вхідних даних (символи, переповнення і т.і.)

Текст програми

```
.model small
JUMPS
.stack 100h

.data
    ; messages
    msg_size      db 'Enter array size (1-20): $'
    msg_elem      db 'Enter element: $'
    msg_sum       db 13, 10, 'Sum of elements: $'
    msg_max       db 13, 10, 'Maximum element: $'
    msg_sort      db 13, 10, 'Sorted array: $'
    msg_space     db ' $'

    msg_err       db 13, 10, 'Error: Invalid input!$'
    msg_ovf       db 13, 10, 'Error: Overflow! Limit to 16-bit.$'
    msg_size_err  db 13, 10, 'Error: Size must be between 1 and 20.$'

    ; variables & array
    buffer        db 7, 0, 7 dup(0)
    arr           dw 20 dup(0)                                ; reserved
for 20 words (40 bytes)
    arr_size      dw 0
    is_neg        db 0

.code
main proc
    mov ax, @data
    mov ds, ax
```

```

; input size
lea dx, msg_size
mov ah, 09h
int 21h

call InputProc
jc exit_prog

; check size [1; 20]
cmp ax, 1
jl bad_size
cmp ax, 20
jg bad_size
mov arr_size, ax

; crlf new line
mov ah, 02h
mov dl, 13 ; \r
int 21h
mov dl, 10 ; \n
int 21h

; input elements
mov cx, arr_size
lea di, arr
input_loop:
push cx ; save counter

lea dx, msg_elem
mov ah, 09h
int 21h

call InputProc
pop cx
jc exit_prog ; invalid input

mov [di], ax ; in c => *di = ax
add di, 2 ; move 1 word ahead

; new line
mov ah, 02h
mov dl, 13
int 21h
mov dl, 10
int 21h

loop input_loop

; find sum
lea dx, msg_sum
mov ah, 09h

```

```

        int 21h

        mov cx, arr_size
        lea si, arr
        xor bx, bx                ; bx - sum

sum_loop:
        mov ax, [si]
        add bx, ax
        jo overflow_err          ; if sum overflows
        add si, 2
        loop sum_loop

        mov ax, bx                ; ax - 1st argument
        call OutputProc

        ; find max
        lea dx, msg_max
        mov ah, 09h
        int 21h

        mov cx, arr_size
        lea si, arr
        mov ax, [si]             ; first max - first element

max_loop:
        cmp ax, [si]
        jge skip_max             ; max >= el - skip
        mov ax, [si]             ; otherwise update max

skip_max:
        add si, 2
        loop max_loop

        call OutputProc

        ; bubble sort
        mov cx, arr_size
        dec cx                   ; num of iters: size - 1
        cmp cx, 0
        je print_sorted          ; only 1 el - skip sort

outer_sort:
        push cx
        mov cx, arr_size
        dec cx
        lea si, arr

inner_sort:
        mov ax, [si]
        mov bx, [si+2]
        cmp ax, bx
        jle no_swap              ; first <= second - skip

        ; swap

```

```

        mov     [si], bx
        mov     [si+2], ax
no_swap:
        add     si, 2
        loop    inner_sort

        pop     cx
        loop    outer_sort

; finally print ts
print_sorted:
        lea     dx, msg_sort
        mov     ah, 09h
        int     21h

        mov     cx, arr_size
        lea     si, arr
print_arr_loop:
        mov     ax, [si]
        push    cx
        push    si
        call    OutputProc

        ; print space between numbers
        lea     dx, msg_space
        mov     ah, 09h
        int     21h

        pop     si
        pop     cx
        add     si, 2
        loop    print_arr_loop

        jmp     exit_prog

bad_size:
        lea     dx, msg_size_err
        mov     ah, 09h
        int     21h
        jmp     exit_prog

overflow_err:
        lea     dx, msg_ovf
        mov     ah, 09h
        int     21h
        jmp     exit_prog

exit_prog:
        mov     ax, 4c00h
        int     21h

```

```
main endp
```

```
; copied from lab2  
; returns in ax
```

```
InputProc proc
```

```
    lea dx, buffer  
    mov ah, 0Ah  
    int 21h  
  
    lea si, buffer+2  
    mov cl, buffer+1  
    xor ch, ch  
    cmp cx, 0  
    je input_error
```

```
    xor bx, bx  
    mov is_neg, 0
```

```
    mov al, [si]  
    cmp al, '-'  
    jne check_plus  
    mov is_neg, 1  
    inc si  
    dec cx  
    jmp parse_loop
```

```
check_plus:
```

```
    cmp al, '+'  
    jne parse_loop  
    inc si  
    dec cx
```

```
parse_loop:
```

```
    cmp cx, 0  
    je parse_done  
  
    mov al, [si]  
    cmp al, '0'  
    jl input_error  
    cmp al, '9'  
    jg input_error  
  
    sub al, '0'  
    xor ah, ah  
    push cx  
  
    mov cx, ax  
    mov ax, bx  
    mov dx, 10
```

```

        imul dx
        jo  ovf_err_proc

        add ax, cx
        jo  ovf_err_proc

        mov bx, ax
        pop cx

        inc si
        dec cx
        jmp parse_loop

parse_done:
        cmp is_neg, 1
        jne save_number
        neg bx
        jo  ovf_err_proc

save_number:
        mov ax, bx
        cld
        ret

ovf_err_proc:
        pop cx
        lea dx, msg_ovf
        mov ah, 09h
        int 21h
        stc
        ret

input_error:
        lea dx, msg_err
        mov ah, 09h
        int 21h
        stc
        ret

InputProc endp

OutputProc proc
        cmp ax, 0
        jge start_convert

        push ax
        mov ah, 02h
        mov dl, '-'
        int 21h
        pop ax
        neg ax

```

```

start_convert:
    xor    cx, cx
    mov    bx, 10

divide_loop:
    xor    dx, dx
    div    bx
    push    dx
    inc    cx
    test    ax, ax
    jnz    divide_loop

print_loop:
    pop    dx
    add    dl, '0'
    mov    ah, 02h
    int    21h
    loop    print_loop

    ret

```

OutputProc endp

end main

Текст другої програми

```

.model small
JUMPS
.stack 100h

.data
    ; messages
    msg_rows    db 'Enter number of rows (1-10): $'
    msg_cols    db 13, 10, 'Enter number of cols (1-10): $'
    msg_elem    db 'Enter element: $'
    msg_search  db 13, 10, 'Enter element to search: $'

    ; result messages
    msg_found   db 13, 10, 'Found at Row: $'
    msg_col     db ', Col: $'
    msg_not_fnd db 13, 10, 'Element not found in the array.$'
    msg_nl      db 13, 10, '$'

    ; error messages
    msg_err     db 13, 10, 'Error: Invalid input! (Numbers only)$'
    msg_ovf     db 13, 10, 'Error: Overflow/Underflow! Limit to 16-
bit.$'
    msg_dim_err db 13, 10, 'Error: Dimension must be between 1 and
10.$'

    ; vars
    buffer      db 7, 0, 7 dup(0)
    arr         dw 100

```

```

dup(0)                                     ; reserved for
10x10 matrix (100 words)
    num_rows    dw 0
    num_cols    dw 0
    search_val  dw 0
    found_flag  db
0                                           ; 1 -
found, 0 - not found
    is_neg      db 0

.code
main proc

    mov ax, @data
    mov ds, ax

    ; rows input
    lea dx, msg_rows
    mov ah, 09h
    int 21h
    call InputProc
    jc  exit_prog
    cmp ax, 1
    jl  bad_dim
    cmp ax, 10
    jg  bad_dim
    mov num_rows, ax

    ; cols input
    lea dx, msg_cols
    mov ah, 09h
    int 21h
    call InputProc
    jc  exit_prog
    cmp ax, 1
    jl  bad_dim
    cmp ax, 10
    jg  bad_dim
    mov num_cols, ax

    ; input elements
    lea dx, msg_nl
    mov ah, 09h
    int 21h

    ; calc all elements
    mov ax, num_rows
    imul num_cols
    mov cx, ax                                ; cx - counter for input

loop

    lea di, arr                                ; di - array start

```

```

input_loop:
    push cx                ; save counter
    lea dx, msg_elem
    mov ah, 09h
    int 21h

    call InputProc
    pop cx
    jc  exit_prog          ; wilted flower

    mov [di], ax           ; save to array
    add di, 2              ; and move to next element

    ; new line, this time a bit better
    lea dx, msg_nl
    mov ah, 09h
    int 21h

    loop input_loop

    ; search input
    lea dx, msg_search
    mov ah, 09h
    int 21h
    call InputProc
    jc  exit_prog
    mov search_val, ax

    ; actual search
    lea di, arr
    mov cx, num_rows       ; outer loop - rows
    mov bx, 1              ; bx - current row
    mov found_flag, 0

row_loop:
    push cx                ; save rows
    mov cx, num_cols       ; inner loop on cols
    mov si, 1              ; si - current col

col_loop:
    mov ax, [di]
    cmp ax, search_val
    jne not_match          ; skip if !=

    ; found at least one
    mov found_flag, 1

    ; save ts
    push ax

```

```

        push bx
        push cx
        push dx
        push si
        push di

        lea dx, msg_found
        mov ah, 09h
        int 21h
        mov ax, bx                ; bx - row number
        call OutputProc

        lea dx, msg_col
        mov ah, 09h
        int 21h
        mov ax, si                ; si - col number
        call OutputProc

        ; restore ts
        pop di
        pop si
        pop dx
        pop cx
        pop bx
        pop ax

not_match:
        add di, 2                ; next element
        inc si                    ; inc col number
        loop col_loop

        inc bx                    ; col end reached - next
row
        pop cx                    ; restore row counter
        loop row_loop

        ; check if found at least one
        cmp found_flag, 1
        je end_prog

        ; if not print a message
        lea dx, msg_not_fnd
        mov ah, 09h
        int 21h

end_prog:
        jmp exit_prog

bad_dim:
        lea dx, msg_dim_err

```

```

        mov     ah, 09h
        int     21h
        jmp     exit_prog

overflow_err:
        lea     dx, msg_ovf
        mov     ah, 09h
        int     21h
        jmp     exit_prog

exit_prog:
        mov     ax, 4c00h
        int     21h
main endp

; copied from lab2
; returns in ax

InputProc proc
        lea     dx, buffer
        mov     ah, 0Ah
        int     21h

        lea     si, buffer+2
        mov     cl, buffer+1
        xor     ch, ch
        cmp     cx, 0
        je      input_error

        xor     bx, bx
        mov     is_neg, 0

        mov     al, [si]
        cmp     al, '-'
        jne     check_plus
        mov     is_neg, 1
        inc     si
        dec     cx
        jmp     parse_loop

check_plus:
        cmp     al, '+'
        jne     parse_loop
        inc     si
        dec     cx

parse_loop:
        cmp     cx, 0

```

```

        je    parse_done

        mov    al, [si]
        cmp    al, '0'
        jnl    input_error
        cmp    al, '9'
        jg     input_error

        sub    al, '0'
        xor    ah, ah
        push   cx

        mov    cx, ax
        mov    ax, bx
        mov    dx, 10
        imul   dx
        jo     ovf_err_proc

        add    ax, cx
        jo     ovf_err_proc

        mov    bx, ax
        pop    cx

        inc    si
        dec    cx
        jmp    parse_loop

parse_done:
        cmp    is_neg, 1
        jne    save_number
        neg    bx
        jo     ovf_err_proc

save_number:
        mov    ax, bx
        cld
        ret

ovf_err_proc:
        pop    cx
        lea    dx, msg_ovf
        mov    ah, 09h
        int    21h
        stc
        ret

input_error:
        lea    dx, msg_err
        mov    ah, 09h

```

```

                                int  21h
                                stc
                                ret

InputProc endp

OutputProc proc

                                cmp  ax, 0
                                jge  start_convert

                                push ax
                                mov  ah, 02h
                                mov  dl, '-'
                                int  21h
                                pop  ax
                                neg  ax

                                start_convert:
                                xor   cx, cx
                                mov  bx, 10

                                divide_loop:
                                xor   dx, dx
                                div  bx
                                push dx
                                inc  cx
                                test ax, ax
                                jnz  divide_loop

                                print_loop:
                                pop  dx
                                add  dl, '0'
                                mov  ah, 02h
                                int  21h
                                loop print_loop

                                ret

OutputProc endp

end main

```

Вміст .lst файлу

Turbo Assembler

Version 4.0

05/23/26 27:52:01

Page

1

lab.asm

```

1    0000                                .model small
2                                      JUMPS
3    0000                                .stack 100h
4
5    0000                                .data
6                                      ; messages
7    0000  45 6E 74 65 72 20 61+  msg_size      db 'Enter
array size (1-20): $'
8          72 72 61 79 20 73 69+
9          7A 65 20 28 31 2D 32+
10         30 29 3A 20 24
11    001A  45 6E 74 65 72 20 65+  msg_elem      db 'Enter
element: $'
12         6C 65 6D 65 6E 74 3A+
13         20 24
14    002A  0D 0A 53 75 6D 20 6F+  msg_sum      db 13, 10,
'Sum of elements: $'
15         66 20 65 6C 65 6D 65+
16         6E 74 73 3A 20 24
17    003E  0D 0A 4D 61 78 69 6D+  msg_max      db 13, 10,
'Maximum element: $'
18         75 6D 20 65 6C 65 6D+
19         65 6E 74 3A 20 24
20    0052  0D 0A 53 6F 72 74 65+  msg_sort      db 13, 10,
'Sorted array: $'
21         64 20 61 72 72 61 79+
22         3A 20 24
23    0063  20 24                                msg_space  db ' $'
24
25    0065  0D 0A 45 72 72 6F 72+  msg_err      db 13, 10,
'Error: Invalid input!$'
26         3A 20 49 6E 76 61 6C+
27         69 64 20 69 6E 70 75+
28         74 21 24
29    007D  0D 0A 45 72 72 6F 72+  msg_ovf      db 13, 10,
'Error: Overflow! Limit to 16-bit.$'

```

```

30          3A 20 4F 76 65 72 66+
31          6C 6F 77 21 20 4C 69+
32          6D 69 74 20 74 6F 20+
33          31 36 2D 62 69 74 2E+
34          24
35  00A1  0D 0A 45 72 72 6F 72+  msg_size_err db 13, 10,
'Error: Size must be between 1      and 20.$'
36          3A 20 53 69 7A 65 20+
37          6D 75 73 74 20 62 65+
38          20 62 65 74 77 65 65+
39          6E 20 31 20 61 6E 64+
40          20 32 30 2E 24
41
42                                     ; variables & array
43  00C9  07 00 07*(00)          buffer          db 7, 0, 7 dup(0)
44  00D2  14*(0000)              arr             dw 20 dup(0)
                                     ; reserved    for 20 words+
45                                     (40 bytes)
46  00FA  0000                  arr_size        dw 0
47  00FC  00                    is_neg          db 0
48
49  00FD                          .code
50  0000                          main proc
51  0000  B8 0000s                mov  ax, @data
52  0003  8E D8                  mov  ds, ax
53
54                                     ; input size
55  0005  BA 0000r                lea  dx, msg_size
56  0008  B4 09                  mov  ah, 09h
57  000A  CD 21                  int  21h

```

2

lab.asm

```
58
59  000C  E8 00FD                call InputProc
60  000F  73 03 E9 00F3            jc  exit_prog
61
62                                ; check size [1; 20]
63  0014  3D 0001                cmp  ax, 1
64  0017  7D 03 E9 00D7            jl  bad_size
65  001C  3D 0014                cmp  ax, 20
66  001F  7E 03 E9 00CF            jg  bad_size
67  0024  A3 00FAr                mov  arr_size, ax
68
69                                ; crlf  new line
70  0027  B4 02                mov  ah, 02h
71  0029  B2 0D                mov  dl, 13      ;
\r
72  002B  CD 21                int  21h
73  002D  B2 0A                mov  dl, 10      ;
\n
74  002F  CD 21                int  21h
75
76                                ; input elements
77  0031  8B 0E 00FAr                mov  cx, arr_size
78  0035  BF 00D2r                lea  di, arr
79  0038                                input_loop:
80  0038  51                push cx          ; save
counter
81
82  0039  BA 001Ar                lea  dx, msg_elem
83  003C  B4 09                mov  ah, 09h
84  003E  CD 21                int  21h
```

```

85
86      0040  E8 00C9                call InputProc
87      0043  59                    pop  cx
88      0044  73 03 E9 00BE          jc   exit_prog
      ; invalid input
89
90      0049  89 05                mov  [di], ax      ;
in c => *di      = ax
91      004B  83 C7 02            add  di, 2          ;
move 1 word      ahead
92
93                                ; new line
94      004E  B4 02                mov  ah, 02h
95      0050  B2 0D                mov  dl, 13
96      0052  CD 21                int  21h
97      0054  B2 0A                mov  dl, 10
98      0056  CD 21                int  21h
99
100     0058  E2 DE                loop input_loop
101
102                                ; find  sum
103     005A  BA 002Ar              lea  dx, msg_sum
104     005D  B4 09                mov  ah, 09h
105     005F  CD 21                int  21h
106
107     0061  8B 0E 00FAr          mov  cx, arr_size
108     0065  BE 00D2r              lea  si, arr
109     0068  33 DB                xor  bx, bx          ;
bx - sum
110     006A                                sum_loop:
111     006A  8B 04                mov  ax, [si]
112     006C  03 D8                add  bx, ax
113     006E  71 03 E9 008A        jo   overflow_err    ;
if sum overflows
114     0073  83 C6 02            add  si, 2

```

3

lab.asm

```
115 0076 E2 F2          loop sum_loop
116
117 0078 8B C3          mov ax, bx          ;
ax - 1st argument
118 007A E8 012B        call OutputProc
119
120                      ; find max
121 007D BA 003Er        lea dx, msg_max
122 0080 B4 09          mov ah, 09h
123 0082 CD 21          int 21h
124
125 0084 8B 0E 00FAr     mov cx, arr_size
126 0088 BE 00D2r        lea si, arr
127 008B 8B 04          mov ax, [si]          ;
first max - first element
128 008D                max_loop:
129 008D 3B 04          cmp ax, [si]
130 008F 7D 05 90 90 90  jge skip_max
    ; max >= el - skip
131 0094 8B 04          mov ax, [si]          ;
otherwise update max
132 0096                skip_max:
133 0096 83 C6 02        add si, 2
134 0099 E2 F2          loop max_loop
135
136 009B E8 010A        call OutputProc
137
138                      ; bubble sort
139 009E 8B 0E 00FAr     mov cx, arr_size
140 00A2 49              dec cx          ; num of
```

```

iters: size - 1
141    00A3  83 F9 00                                cmp    cx, 0
142    00A6  74 25 90 90 90                          je     print_sorted
        ; only 1 el -      skip sort
143
144    00AB                                outer_sort:
145    00AB  51                                push   cx
146    00AC  8B 0E 00FAr                      mov    cx, arr_size
147    00B0  49                                dec     cx
148    00B1  BE 00D2r                          lea     si, arr
149    00B4                                inner_sort:
150    00B4  8B 04                                mov     ax, [si]
151    00B6  8B 5C 02                          mov     bx, [si+2]
152    00B9  3B C3                            cmp     ax, bx
153    00BB  7E 08 90 90 90                    jle     no_swap
        ; first <= second - skip
154
155                                ; swap
156    00C0  89 1C                                mov     [si], bx
157    00C2  89 44 02                          mov     [si+2], ax
158    00C5                                no_swap:
159    00C5  83 C6 02                          add     si, 2
160    00C8  E2 EA                            loop    inner_sort
161
162    00CA  59                                pop     cx
163    00CB  E2 DE                            loop    outer_sort
164
165                                ; finally print ts
166    00CD                                print_sorted:
167    00CD  BA 0052r                          lea     dx, msg_sort
168    00D0  B4 09                            mov     ah, 09h
169    00D2  CD 21                            int     21h
170
171    00D4  8B 0E 00FAr                      mov     cx, arr_size

```

4

lab.asm

```
172  00D8  BE 00D2r                lea  si, arr
173  00DB                                print_arr_loop:
174  00DB  8B 04                mov  ax, [si]
175  00DD  51                push cx
176  00DE  56                push si
177  00DF  E8 00C6            call OutputProc
178
179                                ; print space between
numbers
180  00E2  BA 0063r            lea  dx, msg_space
181  00E5  B4 09            mov  ah, 09h
182  00E7  CD 21            int  21h
183
184  00E9  5E                pop  si
185  00EA  59                pop  cx
186  00EB  83 C6 02          add  si, 2
187  00EE  E2 EB            loop print_arr_loop
188
189  00F0  EB 15 90          jmp  exit_prog
190
191
192  00F3                                bad_size:
193  00F3  BA 00A1r            lea  dx, msg_size_err
194  00F6  B4 09            mov  ah, 09h
195  00F8  CD 21            int  21h
196  00FA  EB 0B 90          jmp  exit_prog
197
198  00FD                                overflow_err:
199  00FD  BA 007Dr            lea  dx, msg_ovf
200  0100  B4 09            mov  ah, 09h
```

```

201    0102    CD 21                                int    21h
202    0104    EB 01 90                            jmp    exit_prog
203
204    0107                                exit_prog:
205    0107    B8 4C00                            mov    ax, 4c00h
206    010A    CD 21                                int    21h
207    010C                                main endp
208
209
210
211                                ; copied from lab2
212                                ; returns in ax
213
214    010C                                InputProc proc
215    010C    BA 00C9r                            lea    dx, buffer
216    010F    B4 0A                            mov    ah, 0Ah
217    0111    CD 21                                int    21h
218
219    0113    BE 00CBr                            lea    si, buffer+2
220    0116    8A 0E 00CAr                        mov    cl, buffer+1
221    011A    32 ED                            xor    ch, ch
222    011C    83 F9 00                            cmp    cx, 0
223    011F    74 7E 90 90 90                    je     input_error
224
225    0124    33 DB                            xor    bx, bx
226    0126    C6 06 00FCr 00                    mov    is_neg, 0
227
228    012B    8A 04                            mov    al, [si]

```

5

lab.asm

```
229  012D  3C 2D                                cmp  al, '-'
230  012F  75 0D 90 90 90                        jne  check_plus
231  0134  C6 06 00FCr 01                        mov  is_neg, 1
232  0139  46                                inc  si
233  013A  49                                dec  cx
234  013B  EB 0A 90                        jmp  parse_loop
235  013E                                check_plus:
236  013E  3C 2B                                cmp  al, '+'
237  0140  75 05 90 90 90                        jne  parse_loop
238  0145  46                                inc  si
239  0146  49                                dec  cx
240
241  0147                                parse_loop:
242  0147  83 F9 00                        cmp  cx, 0
243  014A  74 34 90 90 90                        je   parse_done
244
245  014F  8A 04                        mov  al, [si]
246  0151  3C 30                        cmp  al, '0'
247  0153  7C 4A 90 90 90                        jl   input_error
248  0158  3C 39                        cmp  al, '9'
249  015A  7F 43 90 90 90                        jg   input_error
250
251  015F  2C 30                        sub  al, '0'
252  0161  32 E4                        xor  ah, ah
253  0163  51                        push cx
254
255  0164  8B C8                        mov  cx, ax
256  0166  8B C3                        mov  ax, bx
257  0168  BA 000A                        mov  dx, 10
258  016B  F7 EA                        imul dx
```

259	016D	70 26 90 90 90		jo	ovf_err_proc
260					
261	0172	03 C1		add	ax, cx
262	0174	70 1F 90 90 90		jo	ovf_err_proc
263					
264	0179	8B D8		mov	bx, ax
265	017B	59		pop	cx
266					
267	017C	46		inc	si
268	017D	49		dec	cx
269	017E	EB C7		jmp	parse_loop
270					
271	0180		parse_done:		
272	0180	80 3E 00FCr 01		cmp	is_neg, 1
273	0185	75 0A 90 90 90		jne	save_number
274	018A	F7 DB		neg	bx
275	018C	70 07 90 90 90		jo	ovf_err_proc
276					
277	0191		save_number:		
278	0191	8B C3		mov	ax, bx
279	0193	F8		clc	
280	0194	C3		ret	
281					
282	0195		ovf_err_proc:		
283	0195	59		pop	cx
284	0196	BA 007Dr		lea	dx, msg_ovf
285	0199	B4 09		mov	ah, 09h

6

lab.asm

```
286  019B  CD 21                      int  21h
287  019D  F9                          stc
288  019E  C3                          ret
289
290  019F                      input_error:
291  019F  BA 0065r                  lea  dx, msg_err
292  01A2  B4 09                      mov  ah, 09h
293  01A4  CD 21                      int  21h
294  01A6  F9                          stc
295  01A7  C3                          ret
296  01A8                      InputProc endp
297
298  01A8                      OutputProc  proc
299  01A8  3D 0000                      cmp  ax, 0
300  01AB  7D 0D 90 90 90              jge
start_convert
301
302  01B0  50                          push ax
303  01B1  B4 02                      mov  ah, 02h
304  01B3  B2 2D                      mov  dl, '-'
305  01B5  CD 21                      int  21h
306  01B7  58                          pop  ax
307  01B8  F7 D8                      neg  ax
308
309  01BA                      start_convert:
310  01BA  33 C9                      xor  cx, cx
311  01BC  BB 000A                    mov  bx, 10
312  01BF                      divide_loop:
313  01BF  33 D2                      xor  dx, dx
314  01C1  F7 F3                      div  bx
```

315	01C3	52		push dx
316	01C4	41		inc cx
317	01C5	85 C0		test ax, ax
318	01C7	75 F6		jnz divide_loop
319				
320	01C9		print_loop:	
321	01C9	5A		pop dx
322	01CA	80 C2 30		add dl, '0'
323	01CD	B4 02		mov ah, 02h
324	01CF	CD 21		int 21h
325	01D1	E2 F6		loop print_loop
326				
327	01D3	C3		ret
328	01D4		OutputProc	endp
329				
330			end main	

7

Symbol Table

Symbol Name	Type	Value
??DATE	Text	"05/23/26"
??FILENAME	Text	"lab"
??TIME	Text	"27:52:01"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
ARR	Word	DGROUP:00D2
ARR_SIZE	Word	DGROUP:00FA
BAD_SIZE	Near	_TEXT:00F3
BUFFER	Byte	DGROUP:00C9
CHECK_PLUS	Near	_TEXT:013E
DIVIDE_LOOP	Near	_TEXT:01BF
EXIT_PROG	Near	_TEXT:0107
INNER_SORT	Near	_TEXT:00B4
INPUTPROC	Near	_TEXT:010C
INPUT_ERROR	Near	_TEXT:019F
INPUT_LOOP	Near	_TEXT:0038

IS_NEG	Byte	DGROUP:00FC
MAIN	Near	_TEXT:0000
MAX_LOOP	Near	_TEXT:008D
MSG_ELEM	Byte	DGROUP:001A
MSG_ERR	Byte	DGROUP:0065
MSG_MAX	Byte	DGROUP:003E
MSG_OVF	Byte	DGROUP:007D
MSG_SIZE	Byte	DGROUP:0000
MSG_SIZE_ERR	Byte	DGROUP:00A1
MSG_SORT	Byte	DGROUP:0052
MSG_SPACE	Byte	DGROUP:0063
MSG_SUM	Byte	DGROUP:002A
NO_SWAP	Near	_TEXT:00C5
OUTER_SORT	Near	_TEXT:00AB
OUTPUTPROC	Near	_TEXT:01A8
OVERFLOW_ERR	Near	_TEXT:00FD
OVF_ERR_PROC	Near	_TEXT:0195
PARSE_DONE	Near	_TEXT:0180
PARSE_LOOP	Near	_TEXT:0147
PRINT_ARR_LOOP	Near	_TEXT:00DB
PRINT_LOOP	Near	_TEXT:01C9
PRINT_SORTED	Near	_TEXT:00CD
SAVE_NUMBER	Near	_TEXT:0191
SKIP_MAX	Near	_TEXT:0096
START_CONVERT	Near	_TEXT:01BA
SUM_LOOP	Near	_TEXT:006A

8

Symbol Table

Groups & Segments

Bit Size Align Combine Class

DGROUP

Group

STACK	16	0100	Para	Stack	STACK
__DATA	16	00FD	Word	Public	DATA
__TEXT	16	01D4	Word	Public	CODE

Вміст .map файлу

Start	Stop	Length	Name	Class
00000H	001D3H	001D4H	__TEXT	CODE
001E0H	002DCH	000FDH	__DATA	DATA
002E0H	003DFH	00100H	STACK	STACK

Program entry point at 0000:0000

Вміст другого .lst файлу

1

lab2.asm

```

1    0000                                .model small
2                                JUMPS
3    0000                                .stack 100h
4
5    0000                                .data
6                                ; messages
7    0000  45 6E 74 65 72 20 6E+  msg_rows      db      'Enter
number of rows (1-10): $'
8                                75 6D 62 65 72 20 6F+
9                                66 20 72 6F 77 73 20+
10                               28 31 2D 31 30 29 3A+
11                               20 24
12    001E  0D 0A 45 6E 74 65 72+  msg_cols      db      13, 10,
'Enter number of cols (1-10): $'
```

```

13          20 6E 75 6D 62 65 72+
14          20 6F 66 20 63 6F 6C+
15          73 20 28 31 2D 31 30+
16          29 3A 20 24
17 003E 45 6E 74 65 72 20 65+ msg_elem db 'Enter
element: $'
18          6C 65 6D 65 6E 74 3A+
19          20 24
20 004E 0D 0A 45 6E 74 65 72+ msg_search db 13, 10,
'Enter element to search: $'
21          20 65 6C 65 6D 65 6E+
22          74 20 74 6F 20 73 65+
23          61 72 63 68 3A 20 24
24
25                                     ; result messages
26 006A 0D 0A 46 6F 75 6E 64+ msg_found db 13, 10,
'Found at Row: $'
27          20 61 74 20 52 6F 77+
28          3A 20 24
29 007B 2C 20 43 6F 6C 3A 20+ msg_col db ', Col:
$'
30          24
31 0083 0D 0A 45 6C 65 6D 65+ msg_not_fnd db 13, 10,
'Element not found in the array.$'
32          6E 74 20 6E 6F 74 20+
33          66 6F 75 6E 64 20 69+
34          6E 20 74 68 65 20 61+
35          72 72 61 79 2E 24
36 00A5 0D 0A 24 msg_nl db 13, 10, '$'
37
38                                     ; error messages
39 00A8 0D 0A 45 72 72 6F 72+ msg_err db 13, 10,
'Error: Invalid input! (Numbers only)$'
40          3A 20 49 6E 76 61 6C+
41          69 64 20 69 6E 70 75+
42          74 21 20 28 4E 75 6D+
43          62 65 72 73 20 6F 6E+
44          6C 79 29 24
45 00CF 0D 0A 45 72 72 6F 72+ msg_ovf db 13, 10,
'Error: Overflow/Underflow! Limit to 16-bit.$'
46          3A 20 4F 76 65 72 66+
47          6C 6F 77 2F 55 6E 64+
48          65 72 66 6C 6F 77 21+
49          20 4C 69 6D 69 74 20+
50          74 6F 20 31 36 2D 62+
51          69 74 2E 24
52 00FD 0D 0A 45 72 72 6F 72+ msg_dim_err db 13, 10,
'Error: Dimension must be between 1 and 10.$'
53          3A 20 44 69 6D 65 6E+
54          73 69 6F 6E 20 6D 75+
55          73 74 20 62 65 20 62+
56          65 74 77 65 65 6E 20+
57          31 20 61 6E 64 20 31+

```

```

58          30 2E 24
59
60          ; vars
61 012A 07 00 07*(00)      buffer      db    7, 0, 7
dup(0)
62 0133 64*(0000)          arr         dw    100 dup(0)
          ; reserved for +
63          10x10 matrix (100 words)
64 01FB 0000              num_rows    dw    0
65 01FD 0000              num_cols    dw    0
66 01FF 0000              search_val   dw    0
67 0201 00                found_flag   db    0
          ; 1 - found, 0 - +
68          not found
69 0202 00                is_neg       db    0
70
71 0203                  .code
72 0000                  main proc
73 0000 B8 0000s          mov ax,      @data
74 0003 8E D8            mov ds,      ax
75
76          ; rows input
77 0005 BA 0000r          lea dx,      msg_rows
78 0008 B4 09            mov ah,      09h
79 000A CD 21            int 21h
80 000C E8 0100          call InputProc
81 000F 73 03 E9 00F6    jc  exit_prog
82 0014 3D 0001          cmp ax,      1
83 0017 7D 03 E9 00DA    jl  bad_dim
84 001C 3D 000A          cmp ax,      10
85 001F 7E 03 E9 00D2    jg  bad_dim
86 0024 A3 01FBr          mov num_rows, ax
87
88          ; cols input
89 0027 BA 001Er          lea dx,      msg_cols
90 002A B4 09            mov ah,      09h
91 002C CD 21            int 21h
92 002E E8 00DE          call InputProc
93 0031 73 03 E9 00D4    jc  exit_prog
94 0036 3D 0001          cmp ax,      1
95 0039 7D 03 E9 00B8    jl  bad_dim
96 003E 3D 000A          cmp ax,      10
97 0041 7E 03 E9 00B0    jg  bad_dim
98 0046 A3 01FDr          mov num_cols, ax
99
100         ; input elements
101 0049 BA 00A5r          lea dx,      msg_n1
102 004C B4 09            mov ah,      09h

```

```

103      004E  CD 21                      int  21h
104
105                                ; calc all elements
106      0050  A1 01FBr                    mov  ax,      num_rows
107      0053  F7 2E 01FDr                  imul num_cols
108      0057  8B C8                        mov  cx,      ax
      ; cx - counter for input      loop
109      0059  BF 0133r                      lea  di,      arr
      ; di - array start
110
111      005C                                input_loop:
112      005C  51                                push cx                      ; save
counter
113      005D  BA 003Er                      lea  dx,      msg_elem
114      0060  B4 09                        mov  ah,      09h

```

3

lab2.asm

```

115      0062  CD 21                      int  21h
116
117      0064  E8 00A8                    call InputProc
118      0067  59                        pop  cx
119      0068  73 03 E9 009D              jc   exit_prog
          ; wilted      flower
120
121      006D  89 05                      mov  [di], ax
          ; save to array
122      006F  83 C7 02                  add  di,      2
          ; and move to next element
123
124                                     ; new line, this time a
bit better
125      0072  BA 00A5r                  lea  dx,      msg_nl
126      0075  B4 09                      mov  ah,      09h
127      0077  CD 21                      int  21h
128
129      0079  E2 E1                      loop input_loop
130
131                                     ; search      input
132      007B  BA 004Er                  lea  dx,
          msg_search
133      007E  B4 09                      mov  ah,      09h
134      0080  CD 21                      int  21h
135      0082  E8 008A                    call InputProc
136      0085  73 03 E9 0080              jc   exit_prog
137      008A  A3 01FFr                  mov  search_val, ax
138
139                                     ; actual      search
140      008D  BF 0133r                  lea  di,      arr
141      0090  8B 0E 01FBr                mov  cx,
          num_rows      ; outer loop - rows
142      0094  BB 0001                    mov  bx,      1
          ; bx - current row
143      0097  C6 06 0201r 00            mov  found_flag,
          0
144
145                                     row_loop:
146      009C  51                        push cx          ; save
rows
147      009D  8B 0E 01FDr                mov  cx,
          num_cols      ; inner loop on cols
148      00A1  BE 0001                    mov  si,      1
          ; si - current col
149
150                                     col_loop:
151      00A4  8B 05                      mov  ax,      [di]

```

152	00A6	3B 06 01FFr	cmp	ax,	
	search_val				
153	00AA	75 2C 90 90 90	jne	not_match	
		; skip if !=			
154					
155					; found at least one
156	00AF	C6 06 0201r 01	mov	found_flag,	
		1			
157					
158					; save ts
159	00B4	50	push	ax	
160	00B5	53	push	bx	
161	00B6	51	push	cx	
162	00B7	52	push	dx	
163	00B8	56	push	si	
164	00B9	57	push	di	
165					
166	00BA	BA 006Ar	lea	dx,	
	msg_found				
167	00BD	B4 09	mov	ah,	09h
168	00BF	CD 21	int	21h	
169	00C1	8B C3	mov	ax,	bx
		; bx - row number			
170	00C3	E8 00E5	call	OutputProc	
171					

4

lab2.asm

```
172    00C6  BA 007Br          lea  dx,      msg_col
173    00C9  B4 09          mov  ah,      09h
174    00CB  CD 21          int  21h
175    00CD  8B C6          mov  ax,      si
        ; si - col number
176    00CF  E8 00D9          call OutputProc
177
178                                ; restore ts
179    00D2  5F          pop  di
180    00D3  5E          pop  si
181    00D4  5A          pop  dx
182    00D5  59          pop  cx
183    00D6  5B          pop  bx
184    00D7  58          pop  ax
185
186    00D8                                not_match:
187    00D8  83 C7 02          add  di,      2
        ; next element
188    00DB  46          inc  si          ; inc
col number
189    00DC  E2 C6          loop col_loop
190
191    00DE  43          inc  bx          ; col
end reached - next row
192    00DF  59          pop  cx          ;
restore row counter
193    00E0  E2 BA          loop row_loop
194
195                                ; check if found at least
one
196    00E2  80 3E 0201r 01          cmp  found_flag,
1
197    00E7  74 0A 90 90 90          je   end_prog
198
199                                ; if not print a
message
200    00EC  BA 0083r          lea  dx,
msg_not_fnd
201    00EF  B4 09          mov  ah,      09h
202    00F1  CD 21          int  21h
203
204    00F3                                end_prog:
205    00F3  EB 15 90          jmp  exit_prog
206
207    00F6                                bad_dim:
208    00F6  BA 00FDr          lea  dx,
msg_dim_err
209    00F9  B4 09          mov  ah,      09h
```

```

210      00FB  CD 21                                int  21h
211      00FD  EB 0B 90                            jmp  exit_prog
212
213      0100                                overflow_err:
214      0100  BA 00CFr                            lea  dx,      msg_ovf
215      0103  B4 09                            mov  ah,      09h
216      0105  CD 21                                int  21h
217      0107  EB 01 90                            jmp  exit_prog
218
219      010A                                exit_prog:
220      010A  B8 4C00                            mov  ax,      4c00h
221      010D  CD 21                                int  21h
222      010F                                main endp
223
224
225
226      ; copied from lab2
227      ; returns in ax
228

```

```

229      010F                      InputProc proc
230      010F  BA 012Ar              lea  dx,      buffer
231      0112  B4 0A              mov  ah,      0Ah
232      0114  CD 21              int  21h
233
234      0116  BE 012Cr              lea  si,      buffer+2
235      0119  8A 0E 012Br          mov  cl,      buffer+1
236      011D  32 ED              xor  ch,      ch
237      011F  83 F9 00            cmp  cx,      0
238      0122  74 7E 90 90 90      je   input_error
239
240      0127  33 DB              xor  bx,      bx
241      0129  C6 06 0202r 00      mov  is_neg, 0
242
243      012E  8A 04              mov  al,      [si]
244      0130  3C 2D              cmp  al,      '-'
245      0132  75 0D 90 90 90      jne  check_plus
246      0137  C6 06 0202r 01      mov  is_neg, 1
247      013C  46                  inc  si
248      013D  49                  dec  cx
249      013E  EB 0A 90            jmp  parse_loop
250
251      0141                      check_plus:
252      0141  3C 2B              cmp  al,      '+'
253      0143  75 05 90 90 90      jne  parse_loop
254      0148  46                  inc  si
255      0149  49                  dec  cx
256
257      014A                      parse_loop:
258      014A  83 F9 00            cmp  cx,      0
259      014D  74 34 90 90 90      je   parse_done
260
261      0152  8A 04              mov  al,      [si]
262      0154  3C 30              cmp  al,      '0'
263      0156  7C 4A 90 90 90      jl   input_error
264      015B  3C 39              cmp  al,      '9'
265      015D  7F 43 90 90 90      jg   input_error
266
267      0162  2C 30              sub  al,      '0'
268      0164  32 E4              xor  ah,      ah
269      0166  51                  push cx
270
271      0167  8B C8              mov  cx,      ax
272      0169  8B C3              mov  ax,      bx
273      016B  BA 000A            mov  dx,      10
274      016E  F7 EA              imul dx
275      0170  70 26 90 90 90      jo   ovf_err_proc

```

276							
277	0175	03 C1				add	ax, cx
278	0177	70 1F 90 90 90				jo	
ovf_err_proc							
279							
280	017C	8B D8				mov	bx, ax
281	017E	59				pop	cx
282							
283	017F	46				inc	si
284	0180	49				dec	cx
285	0181	EB C7				jmp	parse_loop

6

lab2.asm

```
286
287 0183 parse_done:
288 0183 80 3E 0202r 01 cmp is_neg, 1
289 0188 75 0A 90 90 90 jne save_number
290 018D F7 DB neg bx
291 018F 70 07 90 90 90 jo
ovf_err_proc
292
293 0194 save_number:
294 0194 8B C3 mov ax, bx
295 0196 F8 clc
296 0197 C3 ret
297
298 0198 ovf_err_proc:
299 0198 59 pop cx
300 0199 BA 00CFr lea dx, msg_ovf
301 019C B4 09 mov ah, 09h
302 019E CD 21 int 21h
303 01A0 F9 stc
304 01A1 C3 ret
305
306 01A2 input_error:
307 01A2 BA 00A8r lea dx, msg_err
308 01A5 B4 09 mov ah, 09h
309 01A7 CD 21 int 21h
310 01A9 F9 stc
311 01AA C3 ret
312 01AB InputProc endp
313
314 01AB OutputProc proc
315 01AB 3D 0000 cmp ax, 0
316 01AE 7D 0D 90 90 90 jge
start_convert
317
318 01B3 50 push ax
319 01B4 B4 02 mov ah, 02h
320 01B6 B2 2D mov dl, '-'
321 01B8 CD 21 int 21h
322 01BA 58 pop ax
323 01BB F7 D8 neg ax
324
325 01BD start_convert:
326 01BD 33 C9 xor cx, cx
327 01BF BB 000A mov bx, 10
328 01C2 divide_loop:
329 01C2 33 D2 xor dx, dx
330 01C4 F7 F3 div bx
331 01C6 52 push dx
```

332	01C7	41		inc	cx	
333	01C8	85 C0		test	ax,	ax
334	01CA	75 F6		jnz	divide_loop	
335						
336	01CC		print_loop:			
337	01CC	5A		pop	dx	
338	01CD	80 C2 30		add	dl,	'0'
339	01D0	B4 02		mov	ah,	02h
340	01D2	CD 21		int	21h	
341	01D4	E2 F6		loop	print_loop	
342						

```
343    01D6    C3                                ret
344    01D7                                OutputProc    endp
345
346                                end main
```

Symbol Name	Type	Value
??DATE	Text	"05/23/26"
??FILENAME	Text	"lab2"
??TIME	Text	"28:01:06"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB2
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
ARR	Word	DGROUP:0133
BAD_DIM	Near	_TEXT:00F6
BUFFER	Byte	DGROUP:012A
CHECK_PLUS	Near	_TEXT:0141
COL_LOOP	Near	_TEXT:00A4
DIVIDE_LOOP	Near	_TEXT:01C2
END_PROG	Near	_TEXT:00F3
EXIT_PROG	Near	_TEXT:010A
FOUND_FLAG	Byte	DGROUP:0201
INPUTPROC	Near	_TEXT:010F
INPUT_ERROR	Near	_TEXT:01A2
INPUT_LOOP	Near	_TEXT:005C
IS_NEG	Byte	DGROUP:0202
MAIN	Near	_TEXT:0000
MSG_COL	Byte	DGROUP:007B
MSG_COLS	Byte	DGROUP:001E
MSG_DIM_ERR	Byte	DGROUP:00FD
MSG_ELEM	Byte	DGROUP:003E
MSG_ERR	Byte	DGROUP:00A8
MSG_FOUND	Byte	DGROUP:006A
MSG_NL	Byte	DGROUP:00A5
MSG_NOT_FND	Byte	DGROUP:0083
MSG_OVF	Byte	DGROUP:00CF
MSG_ROWS	Byte	DGROUP:0000
MSG_SEARCH	Byte	DGROUP:004E
NOT_MATCH	Near	_TEXT:00D8
NUM_COLS	Word	DGROUP:01FD
NUM_ROWS	Word	DGROUP:01FB
OUTPUTPROC	Near	_TEXT:01AB

OVERFLOW_ERR	Near	_TEXT:0100
OVF_ERR_PROC	Near	_TEXT:0198
PARSE_DONE	Near	_TEXT:0183
PARSE_LOOP	Near	_TEXT:014A
PRINT_LOOP	Near	_TEXT:01CC
ROW_LOOP	Near	_TEXT:009C
SAVE_NUMBER	Near	_TEXT:0194
SEARCH_VAL	Word	DGROUP:01FF
START_CONVERT	Near	_TEXT:01BD

Groups & Segments

Bit Size Align Combine Class

DGROUP

Group

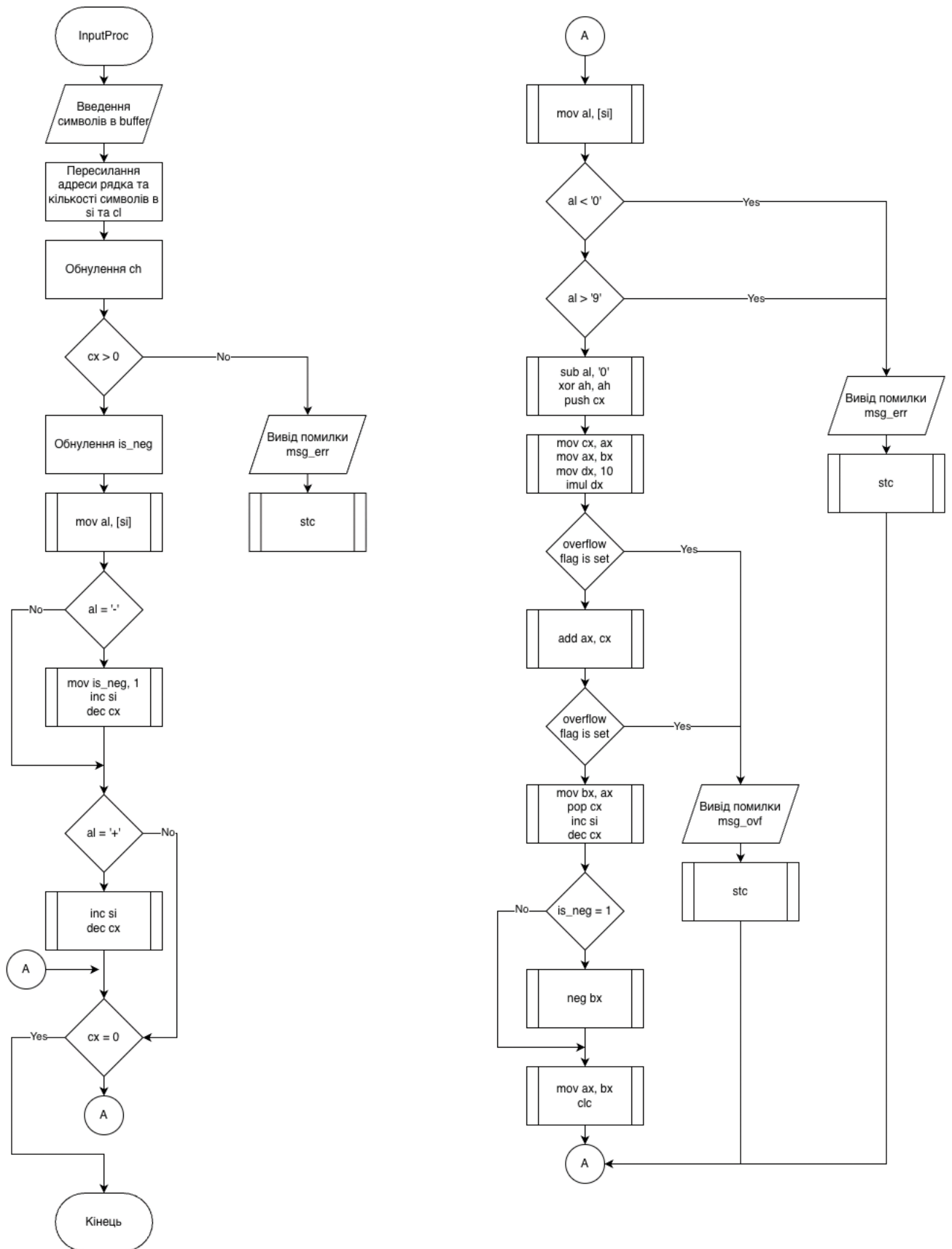
STACK	16	0100	Para	Stack	STACK
DATA	16	0203	Word	Public	DATA
TEXT	16	01D7	Word	Public	CODE

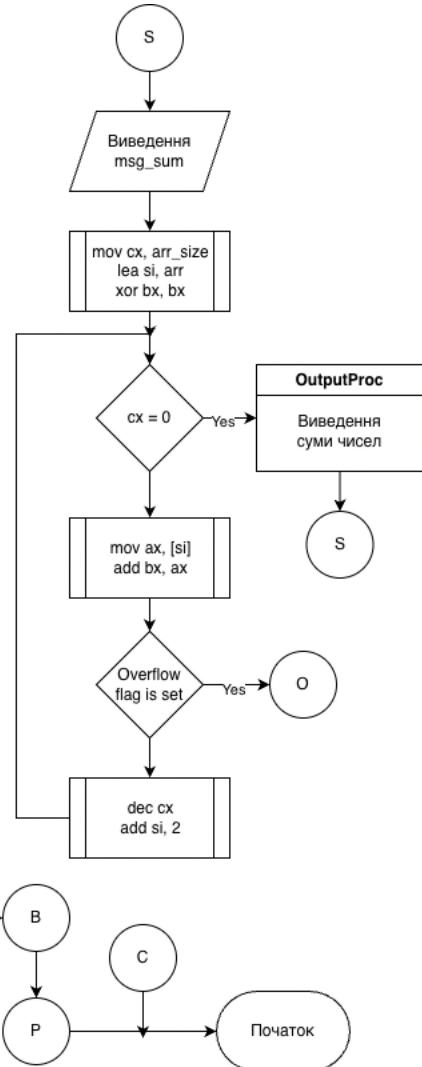
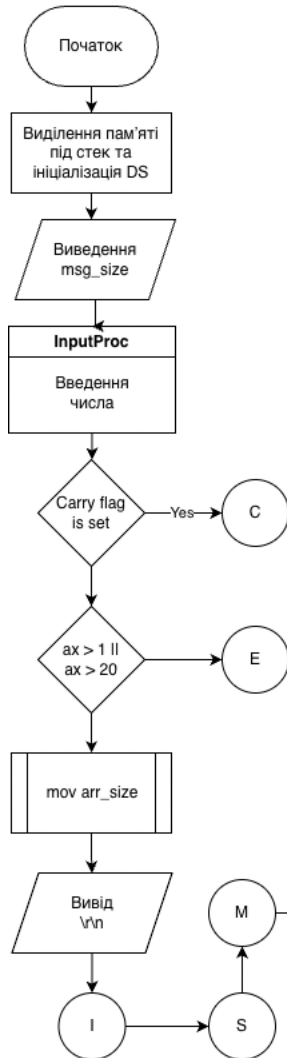
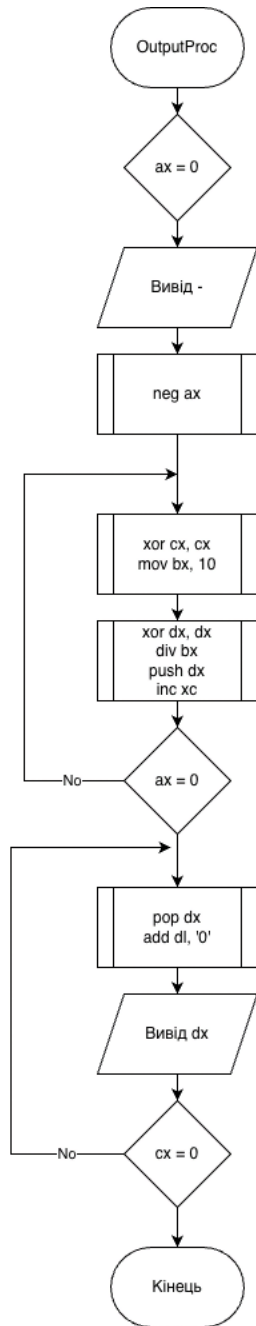
Вміст другого .map файлу

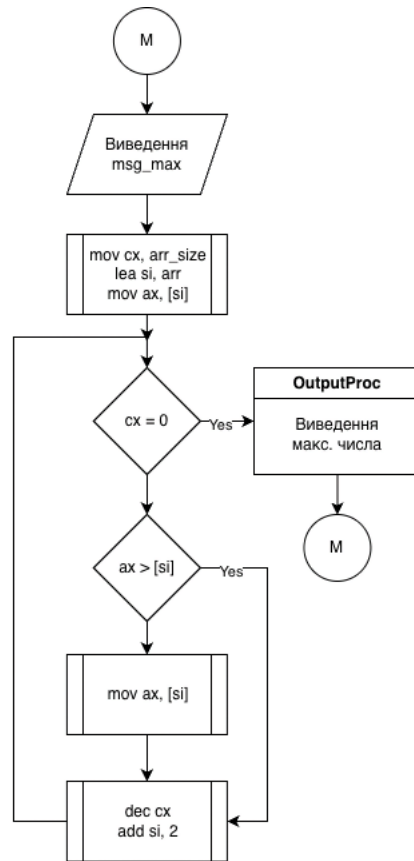
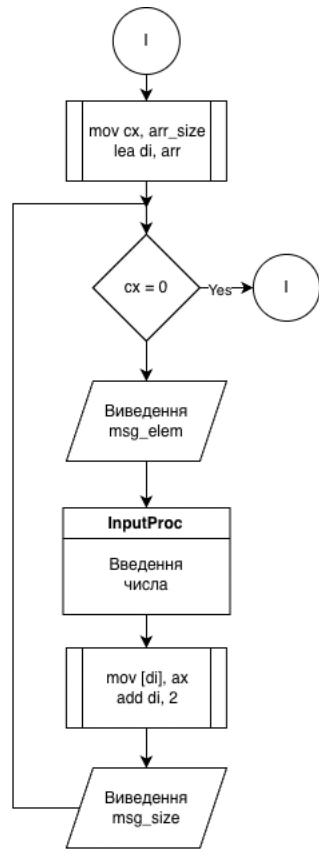
Start	Stop	Length	Name	Class
00000H	001D6H	001D7H	TEXT	CODE
001E0H	003E2H	00203H	DATA	DATA
003F0H	004EFH	00100H	STACK	STACK

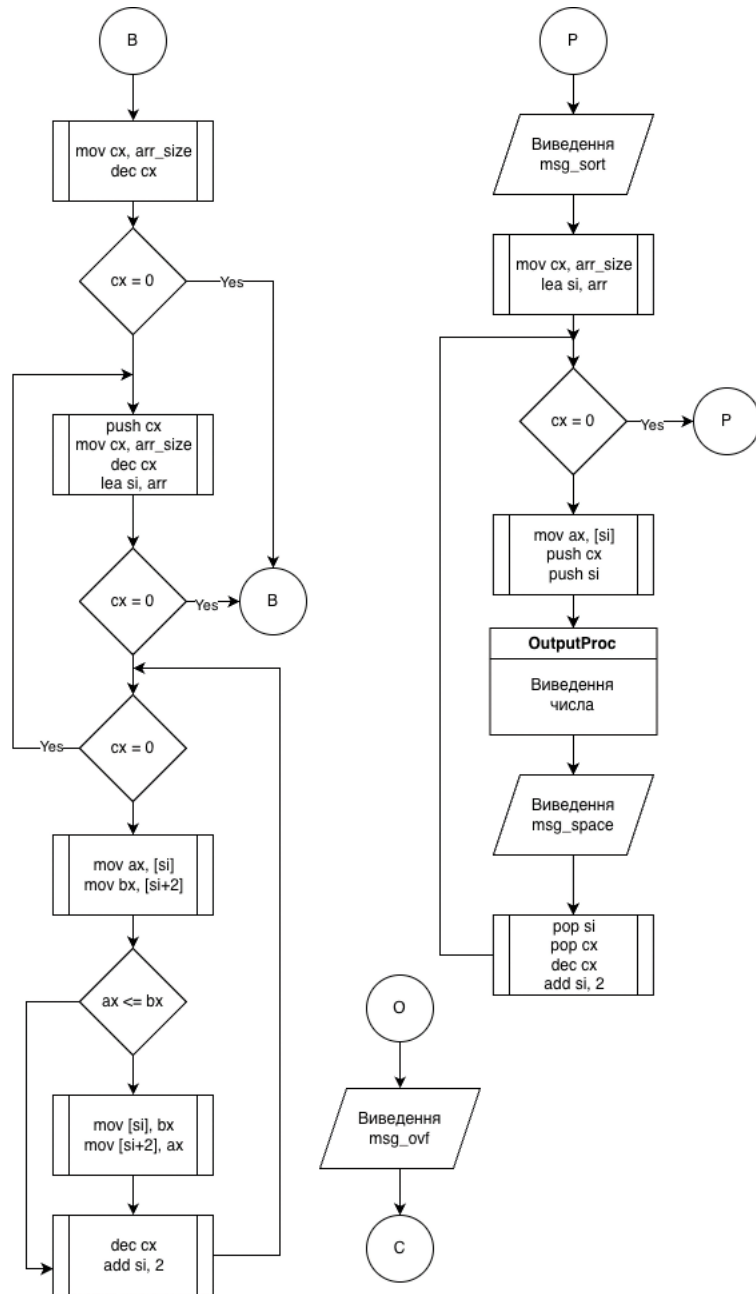
Program entry point at 0000:0000

Блок схема програми









Приклад роботи програм

```
C:\LAB4>LAB.EXE
Enter array size (1-20): 5
Enter element: 5
Enter element: 4
Enter element: 7
Enter element: 8
Enter element: 2

Sum of elements: 26
Maximum element: 8
Sorted array: 2 4 5 7 8
C:\LAB4>LAB2.EXE
Enter number of rows (1-10): 2
Enter number of cols (1-10): 3
Enter element: 6
Enter element: 7
Enter element: 3
Enter element: 4
Enter element: 6
Enter element: 9

Enter element to search: 6
Found at Row: 1, Col: 1
Found at Row: 2, Col: 2
```

Висновок

1. Реалізовано програму яка має можливість введення розміру та елементи одновимірного масиву, знаходження їх суми, максимального елемента та сортування масиву.
2. Реалізовано програму яка має можливість введення розміру та елементи двовимірного масиву та знаходження всіх входжень заданого елемента.
3. Програма має захист від некоректного введення вхідних даних.